

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:30:14.5143589Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-083014515.prof
Profil mémoire	memory-20260924-083014515.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	123.49 ms	123491068.8	0	0

Détail hyperfine

Métrique	Valeur
Moyenne	2.7358 s
Écart-type	0.6531 s
Médiane	2.3070 s
Temps utilisateur	2.0084 s
Temps système	0.8094 s
Min	2.2285 s
Max	3.4720 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	1190.00ms
Mémoire allouée (pprof)	17828.98kB
Mémoire par opération	0 B/op

Garbage collector Go

Cycles GC	56
Temps GC cumulé	63.280 ms

Plus longue pause	6.820 ms
Temps CPU GC cumulé	326.610 ms
Objectif mémoire maximal	97.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **1190.00ms**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	1070ms	89.92%	1150ms	96.64%
jeu-de-la-vie/game.(*Board).index (inline)	60ms	5.04%	60ms	5.04%
jeu-de-la-vie/game.(*Board).Step	20ms	1.68%	1170ms	98.32%
jeu-de-la-vie/game.squaredDistance (inline)	20ms	1.68%	20ms	1.68%
runtime.cgocall	20ms	1.68%	20ms	1.68%
internal/syscall/windows/registry.Key.GetMUISt	0	0.00%	20ms	1.68%
internal/syscall/windows/registry.regLoadMUISt	0	0.00%	20ms	1.68%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	1170ms	98.32%

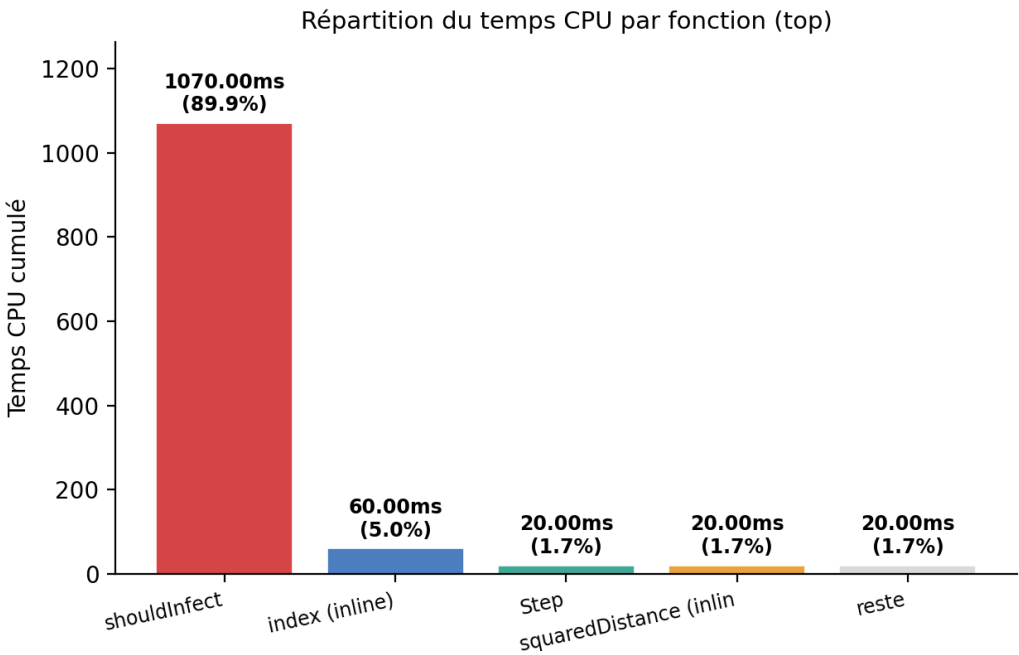


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

File: game.test.exe
 Build ID: C:\Users\saiou\JeuDeLaVie\game.test.exe2026-09-24 10:27:58.0173857 +0200 CEST
 Type: cpu
 Time: 2026-09-24 10:30:14 CEST
 Duration: 1.19s, Total samples = 1190ms (100.13%)
 Showing nodes accounting for 1190ms, 100% of 1190ms total
 Showing top 15 nodes out of 30
 See <https://git.io/JtYMW> for how to read the graph

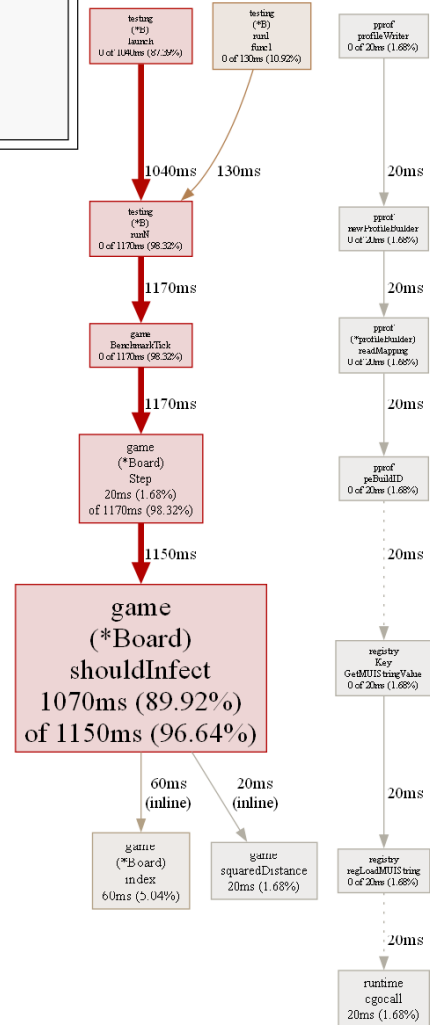


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **89.9%** du temps CPU total (flat), pour **96.6%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

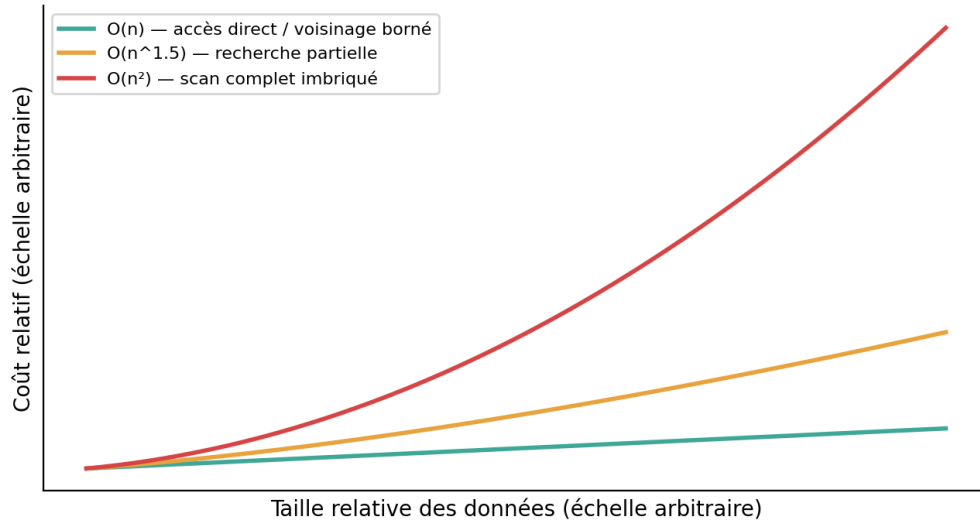


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **17828.98kB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard (inline)	6875.17kB	38.56%	6875.17kB	38.56%
jeu-de-la-vie/game.RandomBoard	5655.11kB	31.72%	12530.28kB	70.28%
runtime/pprof.StartCPUProfile	1184.27kB	6.64%	1184.27kB	6.64%
compress/flate.NewWriter (inline)	902.59kB	5.06%	1553.21kB	8.71%
compress/flate.(*compressor).init	650.62kB	3.65%	650.62kB	3.65%
runtime.malloccg	513kB	2.88%	513kB	2.88%
encoding/base64.init	512.17kB	2.87%	512.17kB	2.87%
time.initLocalFromTZI	512.03kB	2.87%	1024.04kB	5.74%

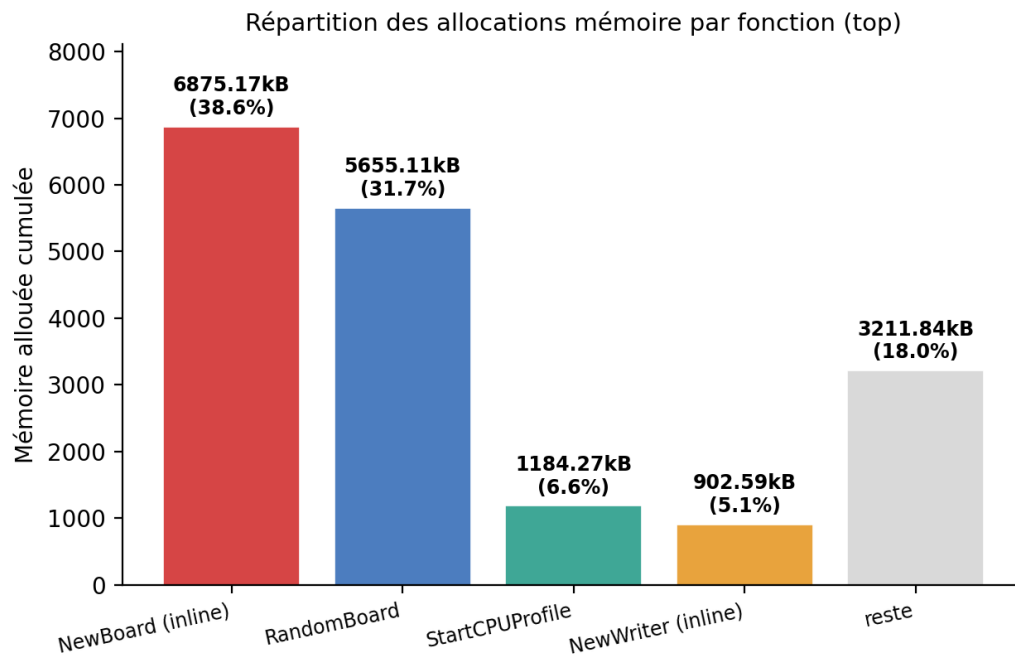


Figure — Répartition des allocations mémoire par fonction.

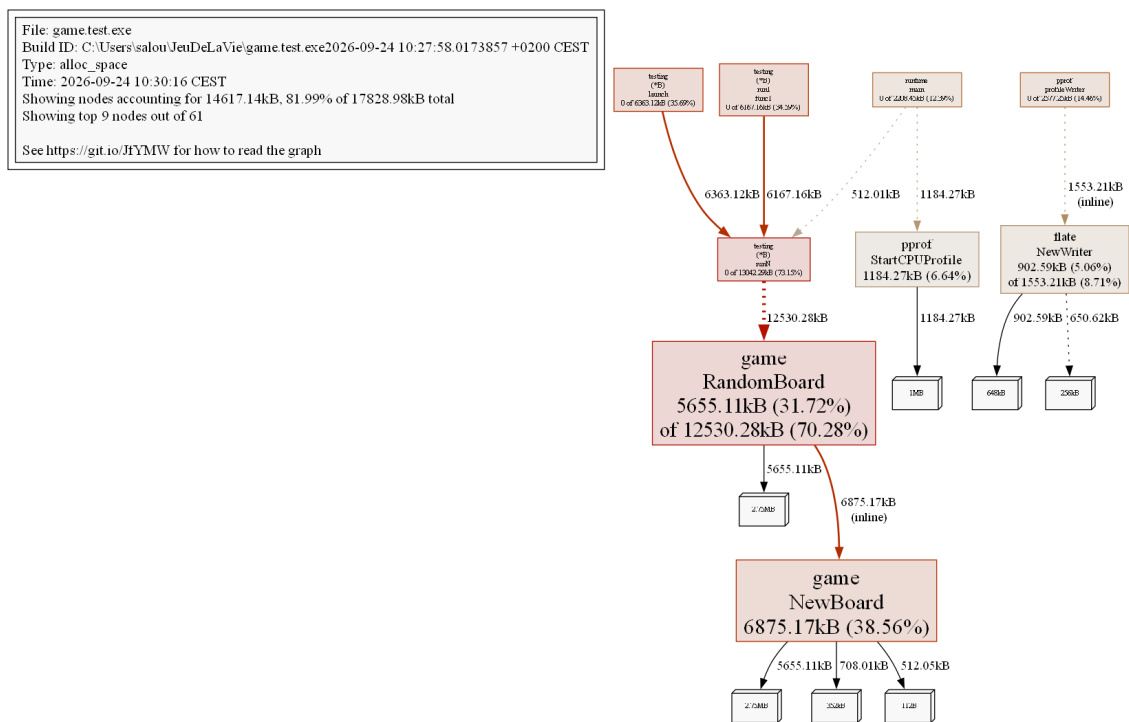


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:30:40 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.